

Nama : Sintia Sari

NIM : 0110222135

Matkul : Machine Learning | Sesi Pagi

Rombel : TI08 - 2022

✓ PRAKTIKUM MANDIRI | DATASET KE-4

```
# Menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')
import os
```

Mounted at /content/gdrive

```
# Memanggil dataset melalui gdrive
path = "gdrive/MyDrive/Machine Learning/Praktikum/Praktikum07/Data/"
```

✓ 1. Loading Dataset

```
# Membaca file csv menggunakan pandas
import pandas as pd

df = pd.read_csv(path + 'dataset_satelit.csv')
df.head()
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma_VV	Sigma_VH	plia	lia	i
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18183	0.04461	35.74446	35.79744	35.41
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.22079	0.04640	35.12096	35.14591	35.41
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.18926	0.03992	35.07724	35.07730	35.41
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.08078	36.08469	35.41
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.18205	0.03797	32.68855	32.69293	35.41

5 rows × 34 columns

✓ 2. Melihat Informasi Data

```
# Melihat informasi data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0   No           594 non-null    int64
1   Longitude    594 non-null    float64
2   Latitude     594 non-null    float64
3   N            594 non-null    float64
4   P            594 non-null    float64
5   K            593 non-null    float64
6   Ca           594 non-null    float64
7   Mg           594 non-null    object
8   Fe           594 non-null    float64
9   Mn           594 non-null    float64
10  Cu           594 non-null    float64
11  Zn           594 non-null    float64
12  B            594 non-null    float64
13  b12          594 non-null    float64
14  b11          594 non-null    float64
15  b9           594 non-null    float64
```

```

16 b8a      594 non-null float64
17 b8      594 non-null float64
18 b7      594 non-null float64
19 b6      594 non-null float64
20 b5      594 non-null float64
21 b4      594 non-null float64
22 b3      594 non-null float64
23 b2      594 non-null float64
24 b1      594 non-null float64
25 Sigma_VV 594 non-null float64
26 Sigma_VH 594 non-null float64
27 plia     594 non-null float64
28 lia     594 non-null float64
29 iafe     594 non-null float64
30 gamma0_vv 594 non-null float64
31 gamma0_vh 594 non-null float64
32 beta0_vv 594 non-null float64
33 beta0_vh 594 non-null float64
dtypes: float64(32), int64(1), object(1)
memory usage: 157.9+ KB

```

Dataset tersebut berisi 34 kolom dan total data sebanyak 594 yang merepresentasikan berbagai parameter lokasi dan kandungan tanah. Kolom **No** adalah nomor urut data, **Longitude** dan **Latitude** menunjukkan posisi geografis. Kolom **N**, **P**, **K**, **Ca**, **Mg**, **Fe**, **Mn**, **Cu**, **Zn**, dan **B** merupakan unsur hara tanah (makro dan mikro). Kolom **b1** hingga **b12** adalah band citra satelit multispektral yang menangkap pantulan cahaya pada panjang gelombang tertentu. Kolom **Sigma_VV**, **Sigma_VH**, **gamma0_vv**, **gamma0_vh**, **beta0_vv**, dan **beta0_vh** adalah parameter radar yang menunjukkan intensitas pantulan gelombang mikro. Sementara **plia**, **lia**, dan **iafe** merupakan parameter tambahan hasil pengolahan citra radar yang berkaitan dengan sudut dan karakteristik pantulan permukaan.

3. Melihat Statistik deskriptif Data

```
df.describe()
```

	No	Longitude	Latitude	N	P	K	Ca	Fe	Mn	Cu	...
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.000000	594.000000	594.000000	...
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.613771	308.034697	2.391195	...
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.579655	241.731643	1.580296	...
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.080000	3.160000	0.090000	...
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.705000	124.015000	1.172500	...
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.650000	239.445000	2.225000	...
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.372500	434.990000	3.357500	...
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.100000	2009.320000	8.170000	...

8 rows × 33 columns

Hasil yang ditampilkan adalah ringkasan statistik dari setiap kolom numerik dalam data (seperti N, P, K, Longitude, dll.). Baris count menunjukkan jumlah data yang tidak kosong di setiap kolom. mean adalah nilai rata-rata, std adalah standar deviasi (sebaran data), sementara min dan max adalah nilai terkecil dan terbesar. Baris 25%, 50% (median), dan 75% menunjukkan kuartil, yang membagi data menjadi empat bagian, membantu melihat distribusi data secara cepat. Intinya, output ini memberikan gambaran umum mengenai pusat, sebaran, dan bentuk dari data.

4. Melihat Jumlah Data Yang Kosong

```
df.isnull().sum()
```

	0
No	0
Longitude	0
Lattitude	0
N	0
P	0
K	1
Ca	0
Mg	0
Fe	0
Mn	0
Cu	0
Zn	0
B	0
b12	0
b11	0
b9	0
b8a	0
b8	0
b7	0
b6	0
b5	0
b4	0
b3	0
b2	0
b1	0
Sigma_VV	0
Sigma_VH	0
plia	0
lia	0
iafe	0
gamma0_vv	0
gamma0_vh	0
beta0_vv	0
beta0_vh	0

dtype: int64

Melihat jumlah data yang kosong dan setelah dilihat keseluruhan dari semua kolom hampir semua kolom tidak memiliki data yang kosong kecuali kolom **K** yang memiliki 1 nilai yang kosong.

5. Import Library Python

```
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

Kode ini menyiapkan lingkungan kerja untuk **analisis data dan pemodelan regresi** dengan mengimpor berbagai *library* penting. **numpy** dan **statsmodels** adalah alat utama untuk **perhitungan numerik dan pembuatan model statistik**. Sementara itu, **matplotlib.pyplot** dan **seaborn** bertanggung jawab untuk **memvisualisasikan** data agar mudah dipahami. Terakhir, *library* **sklearn** menyediakan fungsi **train_test_split** untuk membagi data secara tepat dan fungsi **mean_squared_error** serta **r2_score** untuk **mengevaluasi seberapa baik kinerja model** regresi yang telah dibuat.

6. Melihat Nama-nama Kolom

```
print(df.columns)

Index(['No', 'Longitude', 'Latitude', 'N', 'P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
      'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh'],
      dtype='object')
```

Kode `print(df.columns)` ini berfungsi untuk melihat semua nama kolom yang ada di dalam dataset atau tabel data (`df`). Hasilnya menampilkan daftar lengkap nama-nama kolom tersebut seperti yang tertera diatas.

7. Menentukan Fitur dan Target, Membagi Train dan Test data

```
X = df[['b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh']]

# Kolom target (label)
y = df['N']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)
```

Kode ini berfungsi untuk menyiapkan data sebelum membuat model prediksi. Pertama, menentukan dua bagian utama dari data seperti Fitur (`X`), yaitu semua kolom yang akan digunakan untuk memprediksi (seperti `b12` sampai `beta0_vh`), dan Target (`y`), yaitu kolom yang ingin kita prediksi (`N`). Setelah itu, fungsi `train_test_split` digunakan untuk membagi data `X` dan `y` menjadi dua bagian yaitu `X_train` dan `y_train` (80% data, untuk melatih model), serta `X_test` dan `y_test` (20% data, untuk menguji seberapa baik model bekerja). Hal ini penting untuk memastikan model yang dibuat dapat bekerja dengan baik pada data yang belum pernah dilihatnya.

8. Melihat Jumlah Data Train dan Test

```
# Menampilkan jumlah total data train dan data test
print(f"Jumlah data train: {len(X_train)}")
print(f"Jumlah data test: {len(X_test)}")
```

```
Jumlah data train: 475
Jumlah data test: 119
```

Kode ini berfungsi untuk memastikan jumlah baris data yang sudah dibagi sebelumnya untuk pelatihan (`train`) dan pengujian (`test`). Fungsi `len()` digunakan untuk menghitung banyaknya data di masing-masing kelompok, yaitu `X_train` dan `X_test`. Hasilnya menunjukkan bahwa model akan dilatih menggunakan 475 data, dan kemudian diuji keandalannya menggunakan 119 data yang terpisah.

9. Menambahkan kolom konstanta

```
X = sm.add_constant(X)
```

Kode `X = sm.add_constant(X)` ini berfungsi untuk menambahkan satu kolom baru berisi angka 1 ke dalam data fitur Anda (`X`). Penambahan kolom "konstanta" ini diperlukan dalam model regresi statistik (yang menggunakan *library* `statsmodels` atau `sm`) karena

kolom ini mewakili intersep atau titik potong (β_0) pada persamaan regresi, yaitu nilai prediksi ketika semua fitur lainnya bernilai nol. Tanpa kolom konstanta ini, model regresi Anda tidak akan lengkap dan hasilnya bisa salah.

10. Membuat dan Melatih Model OLS

```
# Membuat dan melatih (fit) model OLS
model_ols = sm.OLS(y_train, X_train).fit()
```

Kode `model_ols = sm.OLS(y_train, X_train).fit()` berfungsi untuk membuat dan melatih model regresi linear. `sm.OLS` (yang berarti Ordinary Least Squares) adalah perintah dari library `statsmodels` yang mendefinisikan jenis model regresi linear yang akan digunakan. Di dalamnya, kita memasukkan data `y_train` (Target/yang diprediksi) dan `X_train` (Fitur/prediktor) yang sudah disiapkan. Kemudian, perintah `.fit()` secara otomatis menjalankan proses perhitungan statistik untuk menemukan garis regresi terbaik yang paling sesuai dengan data pelatihan tersebut. Hasilnya disimpan dalam variabel `model_ols`, yang siap digunakan untuk melihat ringkasan statistik dan melakukan prediksi.

11. Ringkasan Hasil Model OLS

```
print("\n--- RINGKASAN MODEL OLS ---")
print(model_ols.summary())
print("-" * 40)
```

```
--- RINGKASAN MODEL OLS ---
                                OLS Regression Results
=====
Dep. Variable:                  N      R-squared (uncentered):          0.988
Model:                        OLS    Adj. R-squared (uncentered):      0.987
Method:                        Least Squares    F-statistic:                1731.
Date:                        Thu, 06 Nov 2025    Prob (F-statistic):          0.00
Time:                        05:48:16          Log-Likelihood:              -22.718
No. Observations:              475          AIC:                        87.44
Df Residuals:                  454          BIC:                        174.9
Df Model:                      21
Covariance Type:               nonrobust
=====
              coef      std err      t      P>|t|      [0.025      0.975]
-----
b12          1.3588      0.893      1.522      0.129      -0.395      3.113
b11         -1.3718      0.756     -1.813      0.070      -2.858      0.115
b9           0.2077      0.084      2.468      0.014      0.042      0.373
b8a         -0.1795      0.085     -2.117      0.035     -0.346     -0.013
b8           0.0382      0.017      2.291      0.022      0.005      0.071
b7           3.6087      0.821      4.396      0.000      1.995      5.222
b6          -3.0661      0.712     -4.304      0.000     -4.466     -1.666
b5          -2.2275      0.787     -2.829      0.005     -3.775     -0.680
b4           2.6210      1.465      1.789      0.074     -0.259      5.500
b3          -2.7077      2.421     -1.118      0.264     -7.465      2.050
b2           1.0677      1.407      0.759      0.448     -1.698      3.833
b1           0.3538      0.559      0.633      0.527     -0.744      1.452
Sigma_VV     -0.8657      1.457     -0.594      0.553     -3.730      1.998
Sigma_VH      0.6105      1.380      0.442      0.658     -2.102      3.323
plia         0.0021      0.106      0.019      0.985     -0.207      0.211
lia         -0.0021      0.106     -0.019      0.985     -0.211      0.207
iafe         0.0553      0.007      7.634      0.000      0.041      0.070
gamma0_vv    -10.7457      2.164     -4.965      0.000    -14.999     -6.492
gamma0_vh    -5.6735      7.655     -0.741      0.459    -20.718      9.371
beta0_vv      8.7461      1.644      5.320      0.000      5.515     11.977
beta0_vh      2.8849      5.798      0.498      0.619     -8.510     14.279
=====
Omnibus:                2.310    Durbin-Watson:                1.945
Prob(Omnibus):           0.315    Jarque-Bera (JB):              2.162
Skew:                    0.163    Prob(JB):                      0.339
Kurtosis:                3.059    Cond. No.                      4.60e+04
=====
```

Notes:

- [1] R^2 is computed without centering (uncentered) since the model does not contain a constant.
- [2] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [3] The condition number is large, 4.6e+04. This might indicate that there are strong multicollinearity or other numerical problems.

Hasil ini adalah ringkasan performa model regresi yang memprediksi variabel N. Secara keseluruhan, model ini sangat baik karena nilai R-squared (0.988) yang sangat tinggi, yang berarti 98.8% variasi nilai N dapat dijelaskan oleh variabel-variabel prediktor (seperti b12, b11, dll.). Pada tabel di bawah, kolom coef menunjukkan seberapa besar pengaruh setiap prediktor terhadap N. Yang paling penting, kolom P>|t| menunjukkan variabel mana yang berpengaruh signifikan, jika angkanya di bawah 0.05 (misalnya pada b11, b8a, b7, b6, dan b5), maka variabel tersebut benar-benar penting dalam memprediksi N. Sementara itu, pengujian lain seperti Omnibus dan Durbin-Watson membantu memastikan asumsi statistik model terpenuhi.

12. Prediksi Menggunakan Data Uji

```
# Melakukan prediksi pada data uji
y_pred = model_ols.predict(X_test)
```

Kode `y_pred = model_ols.predict(X_test)` ini berfungsi untuk menguji kemampuan model yang sudah dilatih. Model regresi OLS (`model_ols`) yang telah dibuat sebelumnya kini diperintahkan untuk melakukan prediksi (`.predict`) terhadap data uji (`X_test`) yang belum pernah dilihatnya. Hasil dari prediksi ini adalah serangkaian nilai estimasi (`y_pred`) yang akan dibandingkan dengan nilai N yang sebenarnya pada data uji (`y_test`) untuk menilai seberapa akurat model tersebut.

13. Menghitung Metrik Evaluasi

```
# Menghitung metrik evaluasi
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print("\n--- EVALUASI MODEL PADA DATA UJI ---")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R-squared (R²): {r2:.4f}")
print("-" * 40)
```

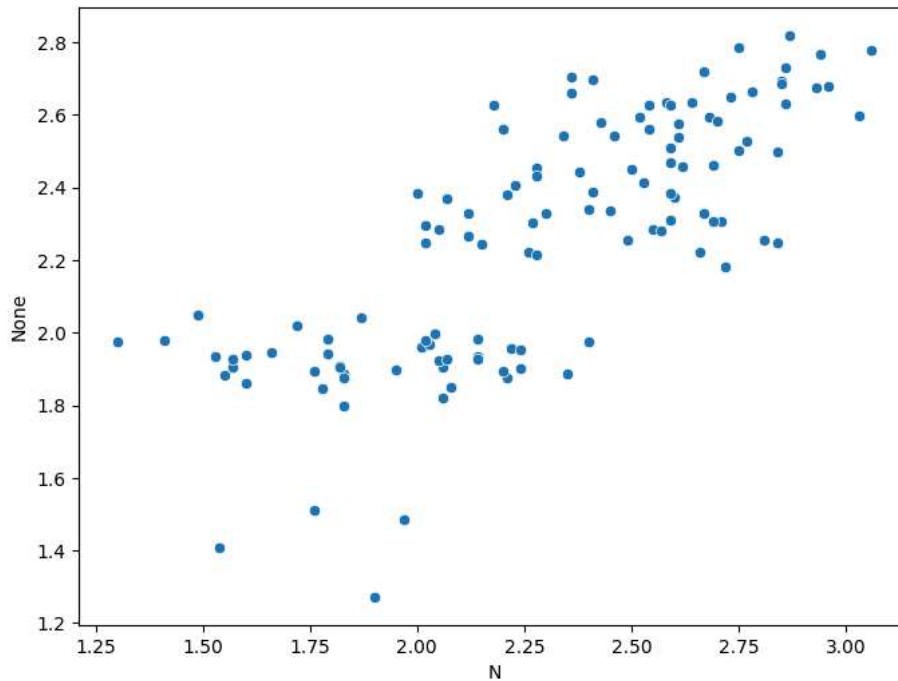
```
--- EVALUASI MODEL PADA DATA UJI ---
Mean Squared Error (MSE): 0.0692
Root Mean Squared Error (RMSE): 0.2630
R-squared (R²): 0.5868
-----
```

Pada bagian ini berfungsi untuk menilai seberapa akurat hasil prediksi model (`y_pred`) dibandingkan dengan nilai sebenarnya pada data uji (`y_test`). Ada tiga metrik yang dihitung yaitu MSE (kesalahan rata-rata kuadrat), RMSE (akar dari MSE), dan R-squared (R^2). Hasilnya menunjukkan R^2 sebesar 0.5868, yang berarti 58.68% variasi nilai target pada data uji dapat dijelaskan oleh model, ini menunjukkan performa model yang cukup baik, namun tidak sempurna pada data pelatihan. Sementara itu, nilai RMSE 0.2630 menunjukkan rata-rata kesalahan prediksi model Anda adalah sekitar 0.2630.

14. Visualisasi Hasil Model

```
plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred)
```

<Axes: xlabel='N', ylabel='None'>



Visualisasi ini adalah diagram sebar (scatter plot) yang membandingkan nilai aktual (N) dengan nilai prediksi model (None) pada data uji. Dari gambar, terlihat bahwa meskipun ada kecenderungan hubungan positif (saat N naik, prediksi juga naik), banyak titik yang menyebar jauh dari garis diagonal imajiner tersebut. Penyebaran ini mengindikasikan bahwa model memiliki beberapa kesalahan prediksi, terutama pada dua kelompok utama data (sekitar nilai 1.8–2.0 dan 2.4–2.8), yang sesuai dengan hasil R^2 sebesar 0.5868 yang menunjukkan kinerja cukup, namun tidak sempurna.

15. Visualisasi Nilai Aktual vs. Prediksi

```
# Menambahkan garis diagonal y=x untuk perbandingan ideal
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
plt.title('Regresi OLS: Nilai Aktual vs. Prediksi (Variabel N)')
plt.xlabel('Nilai N Aktual')
plt.ylabel('Nilai N Prediksi')
plt.grid(True)
plt.show()
```

